

Inverse Kinematics

Phase 4 — 18-DOF Humanoid Robot | ESP32 + micro-ROS + ROS2 Jazzy

Abbous Allouch — INSAT, Génie Logiciel 2024-2025

1. What is Inverse Kinematics?

Kinematics describes the motion of a robot without considering forces. There are two directions to the same problem:

	Forward Kinematics (FK)	Inverse Kinematics (IK)
Question	Given joint angles → where is the foot?	Given foot target → what are the joint angles?
Difficulty	Easy. Always one answer.	Hard. May have 0, 1, or 2 answers.
Used when	Checking / simulating a pose	Walking: controller says 'put foot HERE'
This project	Verification only	100 Hz control loop — Phase 4 node

In walking, the ZMP preview controller (Phase 5) outputs desired CoM and foot positions every 10 ms. The IK node converts those positions into the 18 servo angles that are sent to the ESP32.

2. Method: Analytical Geometric IK

Three methods exist for solving IK:

- Analytical geometric — direct trigonometric formula. Fast, exact, deterministic. Used in this project.
- Jacobian numerical — iterative matrix inversion. General but slow; fails at singularities.
- Neural network — learned mapping. Flexible but non-deterministic; used in Phase 7 (RL layer).

Why analytical? The robot has a fixed known structure. An exact closed-form solution exists and runs in under 1 μ s — well within the 10 ms control budget.

3. Robot Geometry Used by IK

Link lengths — Winter (2009) anthropometric model, scaled to H = 500 mm:

Parameter	Symbol	Value	Used in
Thigh length	L1	0.1225 m	Sagittal IK
Shank length	L2	0.1230 m	Sagittal IK
L1 + L2 (leg reach)	L_leg	0.2455 m	Frontal IK + singularity guard
Hip separation (half)	hip_w / 2	0.04 m	Frame transform
CoM height (LIPM z_c)	com_height	0.19 m	Frame transform

4. Decoupling: Two Independent 2D Problems

A 5-DOF leg in 3D space is complex. The key insight is that for a humanoid walking on flat ground, the sagittal and frontal planes are nearly independent at low speeds:

Plane	Joints solved	Physical meaning
Sagittal (side view)	hip_pitch, knee_pitch, ankle_pitch	Forward / backward motion. How much the leg reaches in front.
Frontal (front view)	hip_roll, ankle_roll	Lateral weight shift. How much the CoM leans left/right.

This turns one hard 3D problem into two simple 2D problems, each solvable with a few lines of trigonometry.

5. Sagittal Plane IK — Step by Step

The hip, knee, and foot form a triangle. All three side lengths are known, so the triangle is fully determined.

5.1 Frame Setup

Inputs (foot position in hip frame):

- x_f = foot forward offset from hip (+ = in front)
- z_f = foot downward offset from hip (+ = toward ground, always positive)

Known constants: $L1 = 0.1225$ m, $L2 = 0.1230$ m

5.2 Step A — Knee Angle (Law of Cosines)

The straight-line distance from hip to foot:

$$d = \sqrt{x_f^2 + z_f^2}$$

The law of cosines applied to the hip-knee-foot triangle:

$$c_k = (L1^2 + L2^2 - d^2) / (2 \cdot L1 \cdot L2)$$

$$q_{knee} = \text{atan2}(\sqrt{1 - c_k^2}, c_k)$$

Why atan2 instead of acos? $\text{atan2}(\sin, \cos)$ handles all quadrants correctly, is numerically stable near ± 1 , and always returns $q_{knee} \geq 0$ (knee bent forward = human-like solution).

5.3 Step B — Hip Pitch

Two angles define the hip pitch:

$$\begin{aligned} \gamma &= \text{atan2}(x_f, z_f) && // \text{ angle of the hip-to-foot vector from vertical} \\ \beta &= \text{atan2}(L2 \cdot \sin(q_{knee}), L1 + L2 \cdot \cos(q_{knee})) && // \text{ angle inside triangle at hip} \\ q_{hip} &= \gamma - \beta \end{aligned}$$

5.4 Step C — Ankle Pitch (Fix Missing from Original Report)

Critical: The original Selmi (2024) report did not include this step. Without it, the foot tilts forward every time the knee bends, and the robot tips over on landing.

The foot must stay horizontal. The sum of all pitch rotations in the leg chain must equal zero:

$$\begin{aligned} q_{hip} + q_{knee} + q_{ankle} &= 0 \\ q_{ankle} &= -(q_{hip} + q_{knee}) \end{aligned}$$

Verification:

- Standing straight ($x_f=0$, $z_f=L_1+L_2$): $q_{\text{knee}}=0^\circ$, $q_{\text{hip}}=0^\circ$, $q_{\text{ankle}}=0^\circ$ ✓
- Walking step ($q_{\text{hip}}=15^\circ$, $q_{\text{knee}}=30^\circ$): $q_{\text{ankle}} = -45^\circ$ (tilts back, foot stays flat) ✓

6. Frontal Plane IK

When the CoM shifts laterally by δy to load a foot, the leg forms a right triangle in the frontal plane:

```
q_hip_roll = atan2(delta_y, L_leg)    // L_leg = L1 + L2 = 0.2455 m
q_ankle_roll = -q_hip_roll           // ankle compensates to keep foot flat
```

Example: CoM shifts 3 cm right $\rightarrow \delta y = +0.03$ m

```
q_hip_roll = atan2(0.03, 0.2455) = 7.0°
q_ankle_roll = -7.0°
```

7. Frame Transformation (World $\rightarrow$ Hip)

The ZMP preview controller publishes positions in the world frame (origin = start, z = up). The IK functions need foot position relative to the hip joint.

Hip position in world frame:

```
hip_R = ( com_x,    com_y + hip_width/2,    com_z )    // right hip
hip_L = ( com_x,    com_y - hip_width/2,    com_z )    // left hip
```

Foot in hip frame (note z-axis flip):

```
xf = foot_x - hip_x    // forward offset (sagittal)
yf = foot_y - hip_y    // lateral offset (frontal)  → delta_y
zf = hip_z - foot_z    // downward offset (sagittal) FLIP: hip minus foot
```

Why flip z ? World frame z points up. IK sagittal plane uses z pointing downward (foot below hip is positive). The subtraction $\text{hip}_z - \text{foot}_z$ performs this conversion.

8. Singularity: The Fully Extended Leg

A singularity occurs when the leg is fully extended ($d = L_1 + L_2 = 0.2455$ m). At this point:

- $q_{\text{knee}} = 0^\circ$ — the triangle collapses to a line
- Infinite combinations of hip and ankle angles produce the same foot position
- The Jacobian matrix becomes rank-deficient — the IK has no unique solution
- Tiny foot position changes cause enormous, violent joint angle changes

The IK node guards against this in two ways:

- $\text{knee_min_deg} = 5.0^\circ$ — enforced minimum knee bend (configured in `ik_params.yaml`)
- $d_{\text{max}} = 0.99 \times (L_1+L_2)$ — foot target is clamped 1% short of full extension with a warning log

This is why the robot always walks in a slightly crouched posture. The rest pose uses $q_{\text{knee}} \approx 100^\circ$ (right) and 70° (left) — well away from the singularity.

9. Geometric Angle $\rightarrow$ Servo Command

Geometric angles (radians, zero = straight leg) must be mapped to servo PWM commands using the rest angles from Table 3.1. Servo angle is in degrees.

Right leg:

Idx	Joint	Formula	Note
5	r_hip_roll	$\theta = 50 + q_hip_roll \times (180/\pi)$	
6	r_hip_pitch	$\theta = 80 + q_hip_pitch \times (180/\pi)$	
7	r_knee_pitch	$\theta = 100 - q_knee \times (180/\pi)$	MINUS — inverted
8	r_ankle_pitch	$\theta = 100 + q_ankle \times (180/\pi)$	
9	r_ankle_roll	$\theta = 90 + q_ankle_roll \times (180/\pi)$	

Left leg (mirror-mounted — roll signs are negated):

Idx	Joint	Formula	Note
13	l_hip_roll	$\theta = 50 - q_hip_roll \times (180/\pi)$	MINUS — mirror
14	l_hip_pitch	$\theta = 80 + q_hip_pitch \times (180/\pi)$	
15	l_knee_pitch	$\theta = 70 - q_knee \times (180/\pi)$	Rest=70, MINUS
16	l_ankle_pitch	$\theta = 90 + q_ankle \times (180/\pi)$	
17	l_ankle_roll	$\theta = 90 - q_ankle_roll \times (180/\pi)$	MINUS — mirror

Left knee asymmetry: The right knee rest is 100° while the left is 70°. This indicates the left knee servo is mounted in the opposite orientation. The minus sign in the formula compensates. Verify physically and flip if needed.

Signs must be verified in Gazebo. After importing the URDF (Phase 6), check each leg joint moves in the correct direction. If a joint moves backward, flip the +/- sign in the servo formula.

10. The IK Node — ROS2 Architecture

Topics

Topic	Type	Dir	Rate	QoS
/com_trajectory	geometry_msgs/PoseStamped	SUB	100 Hz	best_effort
/foot_trajectory	geometry_msgs/PoseArray	SUB	100 Hz	best_effort
/joint_commands	std_msgs/Float32MultiArray	PUB	100 Hz	reliable

PoseArray convention for /foot_trajectory

- poses[0] = right foot (y = +hip_width/2 when standing)

- `poses[1]` = left foot ($y = -\text{hip_width}/2$ when standing)

Control loop (100 Hz timer)

- Read latest `com_x/y/z` and foot positions from callbacks
- Compute hip world positions from CoM + hip offsets
- Transform foot positions from world frame to hip frame
- Call `solve_sagittal()` for each leg $\rightarrow$ `q_hip`, `q_knee`, `q_ankle`
- Call `solve_frontal()` for each leg $\rightarrow$ `q_hip_roll`, `q_ankle_roll`
- Apply servo offset formulas for all 10 leg joints
- Clamp all 18 joints to limits from Table 3.1
- Publish `Float32MultiArray[18]` to `/joint_commands`

QoS reasoning

- Trajectory subscribers use `best_effort`, `depth=1`: always process the latest frame, never queue stale ones. At 100 Hz a dropped frame is replaced in 10 ms.
- `/joint_commands` uses `reliable`: servo commands must not be silently lost — the ESP32 must receive every update.

11. Parameters (`ik_params.yaml`)

Parameter	Default	Unit	Description
L1	0.1225	m	Thigh length
L2	0.1230	m	Shank length
hip_width	0.0800	m	Left-to-right hip joint distance
com_height	0.1900	m	LIPM z_c — must match ZMP controller
knee_min_deg	5.0	°	Minimum knee bend — singularity guard
enabled	true	bool	false $\rightarrow$ publish rest pose only (safe testing)

All parameters can be changed at runtime without restarting the node:

```
ros2 param set /inverse_kinematics enabled false
ros2 param set /inverse_kinematics knee_min_deg 10.0
```

12. File Structure

File	Purpose
<code>src/inverse_kinematics.cpp</code>	Main IK node — all math, ROS2 pub/sub, 100 Hz timer
<code>config/ik_params.yaml</code>	Robot geometry and safety parameters
<code>launch/walking.launch.py</code>	Launches <code>imu_filter</code> + <code>balance_controller</code> + <code>inverse_kinematics</code>
<code>test_ik.py</code>	Manual trajectory publisher to verify IK output

13. Build, Launch, and Test Commands

Build

```
cd ~/Abbes/ros2_ws
colcon build --packages-select humanoid_robot
source install/setup.bash
```

Launch full walking stack

```
ros2 launch humanoid_robot walking.launch.py ik_enabled:=true
```

Launch with IK disabled (safe rest-pose mode)

```
ros2 launch humanoid_robot walking.launch.py ik_enabled:=false
```

Verify the node and its topics

```
ros2 node info /inverse_kinematics
ros2 topic hz /joint_commands          # should print ~100 Hz
```

Manual trajectory injection for testing

```
ros2 topic pub /com_trajectory geometry_msgs/msg/PoseStamped \
  '{pose: {position: {x: 0.0, y: 0.0, z: 0.19}}}' --rate 10

ros2 topic pub /foot_trajectory geometry_msgs/msg/PoseArray \
  '{poses: [{position: {x:0.0, y:0.04, z:0.0}},
    {position: {x:0.0, y:-0.04, z:0.0}}]}' --rate 10
```

Test script (requires IK node running)

```
python3 ~/Abbes/test_ik.py standing
python3 ~/Abbes/test_ik.py lean_left
python3 ~/Abbes/test_ik.py step_right
python3 ~/Abbes/test_ik.py reachable
```

Live parameter tuning (no restart needed)

```
ros2 param set /inverse_kinematics enabled false
ros2 param set /inverse_kinematics knee_min_deg 8.0
ros2 param dump /inverse_kinematics > ik_tuned.yaml
```

14. Integration with the Full Walking Stack

The IK node sits between the trajectory generators and the hardware:

Source	Topic / Data	Consumer
--------	--------------	----------

ZMP preview controller (Ph.5)	→	/com_trajectory	→	inverse_kinematics node
Gait generator (Ph.5)	→	/foot_trajectory	→	inverse_kinematics node
inverse_kinematics node	→	/joint_commands (18 floats)	→	ESP32 → PCA9685 → 18× MG996R
balance_controller (Ph.3)	→	/joint_commands (ankle corrections)	→	ESP32 (PID layer on top of IK)

Both the IK node and the balance controller publish to /joint_commands simultaneously. The ESP32 processes whichever message arrives latest. In practice the PID balance controller adds small ankle corrections on top of the planned IK trajectory, providing real-time disturbance rejection while walking.

15. Known Limitations and Future Work

Limitation	Impact	Fix / Phase
Sagittal/frontal coupling ignored	Small position error at speed	Acceptable at slow walking; fix at Phase 6
Frontal IK uses constant L_leg	Slight lateral vertical bob	Account for actual leg geometry in Phase 6
Servo signs not yet verified	Joint may move wrong direction	Verify in Gazebo — Phase 6
No orientation control (foot angle)	Foot stays horizontal only	Sufficient for flat ground walking
Open-loop IK	No feedback; falls on disturbance	Phase 3 PID runs in parallel; Phase 7 RL

16. References

- Selmi (2024) — prior work: hardware design, basic gait, servo mapping
- Kajita et al. (2003) — ZMP preview control for biped walking (Phase 5)
- Winter D.A. (2009) — Biomechanics and Motor Control of Human Movement — anthropometric proportions
- Hernandez-Santos et al. (2012) — Kinematics and Dynamics of a 16-DOF Humanoid Biped (Journal of Bionic Engineering) — sagittal/frontal decoupling
- IOSR-JMCE — Inverse Kinematics Solution for Biped Robot — geometric approach for 5-link biped
- Robot Academy / Peter Corke — Inverse Kinematics for a 2-joint robot arm using geometry (lecture series)